

2026年9月7日 · 第3周

系统开发工具基础

第3周实验与课后拓展报告

Python 打包 · 智能体编程 · 工程协作 · PyTorch

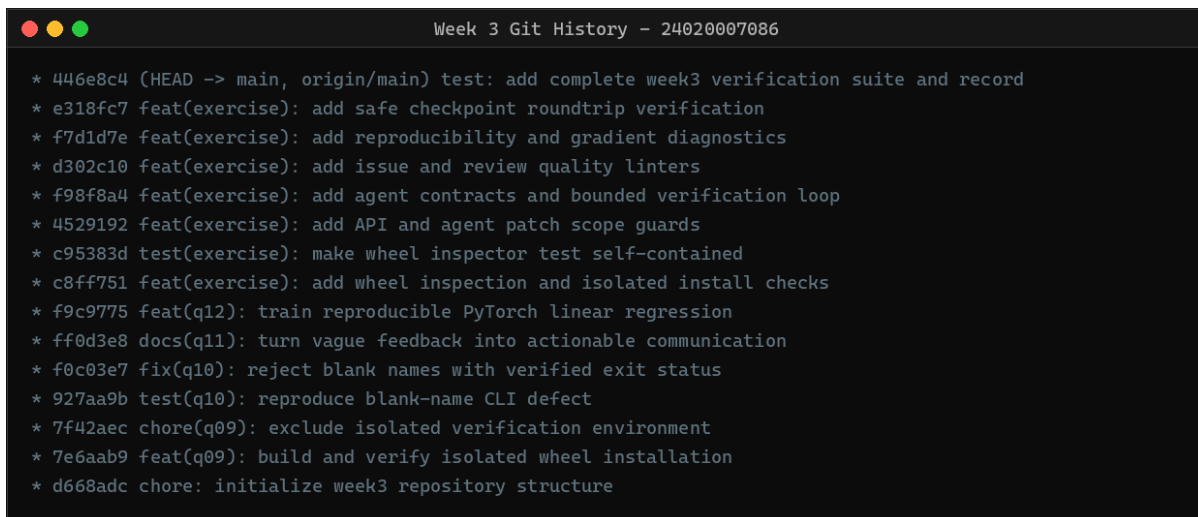
姓名	罗钟杰	学号	24020007086
课程	系统开发工具基础	指导教师	周小伟
学校	中国海洋大学	报告日期	2026年9月7日
实验日期	2026年9月7日	实验主题	第3周工程交付实践

GitHub 仓库 <https://github.com/luojierochester/System-Dev-Tools-Week3>

1. 实验概述与仓库架构

第3周实验覆盖从源码到可安装产物、从失败测试到最小修复、从模糊表述到可执行协作信息，以及从随机初始化到可复现训练结果的完整工程链路。课堂部分完成q09–q12；课后在exercises/下完成q13_ext–q23_ext共11个实例。仓库地址为[System-Dev-Tools-Week3](#)。

目录按职责划分：课堂任务位于q09/–q12/，课后拓展位于exercises/；统一验证入口为scripts/verify_all.ps1，运行记录与报告分别保存在docs/和output/pdf/。提交过程按题目和每1–2个拓展实例拆分，q10还单独保留失败测试与修复提交，使“问题存在”和“修复有效”都有历史证据。



```
Week 3 Git History - 24020007086

* 446e8c4 (HEAD -> main, origin/main) test: add complete week3 verification suite and record
* e318fc7 feat(exercise): add safe checkpoint roundtrip verification
* f7d1d7e feat(exercise): add reproducibility and gradient diagnostics
* d302c10 feat(exercise): add issue and review quality linters
* f98f8a4 feat(exercise): add agent contracts and bounded verification loop
* 4529192 feat(exercise): add API and agent patch scope guards
* c95383d test(exercise): make wheel inspector test self-contained
* c8ff751 feat(exercise): add wheel inspection and isolated install checks
* f9c9775 feat(q12): train reproducible PyTorch linear regression
* ff0d3e8 docs(q11): turn vague feedback into actionable communication
* f0c03e7 fix(q10): reject blank names with verified exit status
* 927aa9b test(q10): reproduce blank-name CLI defect
* 7f42aec chore(q09): exclude isolated verification environment
* 7e6aab9 feat(q09): build and verify isolated wheel installation
* d668adc chore: initialize week3 repository structure
```

图 1: 第3周仓库的原子化提交图谱

仓库核验

图 1 展示初始化、四项课堂实现、六组课后练习和统一验证提交。提交信息使用Conventional Commits，且本地main与远程origin/main保持同步。

2. 课堂任务实现（q09–q12）

2.1 q09: 从源码构建并在干净环境安装Wheel

2.1.1 任务分析

该题采用src/ 布局，发行名替换为greetlab-24020007086，并通过project.scripts把sdt-greet映射到greetlab.cli:main。验证时先运行python -m build，再把生成的Wheel安装到独立虚拟环境，并切换到q09目录之外执行命令，排除“直接导入当前源码”的假通过。

代码清单1: q09 的构建元数据与入口

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-24020007086"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
```

执行结果

构建生成greetlab_24020007086-0.1.0-py3-none-any.whl。元数据检查得到名称greetlab-24020007086、标签py3-none-any和入口sdt-greet = greetlab.cli:main。干净环境安装后，在源码目录外运行得到Hello, 24020007086!。

2.2 q10: 让编程智能体进入可验证的修复循环

2.2.1 任务分析

首先复制q09并添加失败测试：纯空白姓名应触发SystemExit(2)。第一次pytest实际输出Hello, !，并报告DID NOT RAISE SystemExit，证明缺陷确实存在。修复时仅在CLI中执行strip()并调用parser.error()；随后增加正常姓名回归测试，检查diff后复测。

代码清单2: q10 的失败断言与最小修复

```
with pytest.raises(SystemExit) as error:
    main()
assert error.value.code == 2

name = args.name.strip()
if not name:
    parser.error("name must not be blank")
print(f"Hello, {name}!")
```

执行结果

红灯提交test(q10): reproduce blank-name CLI defect 明确保存失败阶段; 修复提交fix(q10): reject blank names with verified exit status 保存绿灯阶段。最终2项pytest全部通过, ai_log.md用4行记录提示、改动和人工验证。

2.3 q11: 把协作材料改成可执行信息

2.3.1 任务分析

原始描述缺少环境、复现步骤、可观察结果和行动条件。重写后的Issue明确记录Windows环境、复现命令、期望退出码2和实际退出码0; 未知Python与包版本标为“待确认”。提交标题采用祈使语气, 正文说明缺陷与解决方案。评审意见标记为Blocking, 并同时给出行为、风险、建议动作和合并条件。

代码清单3: q11的核心协作信息

```
Issue: Reject blank names
Environment: Windows; versions pending confirmation
Reproduce: sdt-greet --name " "
Expected: stderr and exit status 2
Actual: Hello, ! and exit status 0
Review: Blocking - validate name.strip() and add a test
```

执行结果

communication.md共163个汉字, 小于400字限制。课后Issue与Review检查器分别输出issue quality: PASS和review quality: PASS, 说明材料不仅可读, 还能被规则验证。

2.4 q12: 修复可复现的线性回归训练循环

2.4.1 任务分析

训练以种子20260907初始化nn.Linear(1,1), 目标为 $y = 3x - 1$ 。每轮严格按“清空梯度、前向计算、反向传播、更新参数”执行; 训练结束切换到eval(), 在no_grad()中计算最终损失。参数完全由SGD学得, 没有直接赋值为3和-1。

代码清单4: q12的训练与评估步骤

```
for _ in range(200):
    prediction = model(x)
    loss = loss_fn(prediction, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y).item()
```

执行结果

CPU 实际运行输出为：

final_loss=0.000000000

weight=2.999997 bias=-1.000000

测试同时检查损失小于0.001 和参数接近目标值，1 项unittest 通过。

3. 课后拓展练习（11 个实例）

课后练习沿着“产物可交付、修改可约束、协作可执行、训练可复现”四条主线展开。每个实例都有README、独立脚本及测试或演示入口，统一由验证脚本执行。

表 1: 第3周课后拓展实例总览

编号	主题	核心技术	主文件
Ex01	Wheel 元数据检查	zipfile, METADATA, 入口	q13_ext/ wheel_inspector.py
Ex02	隔离安装冒烟测试	venv, pip -no-index	q14_ext/ verify_wheel.py
Ex03	API 兼容性门禁	AST、公共符号差集	q15_ext/ api_guard.py
Ex04	补丁范围门禁	路径归一化、允许列表	q16_ext/ patch_scope_guard.py
Ex05	智能体任务契约	JSON、目标、约束、测试	q17_ext/ prompt_contract.py
Ex06	有界验证循环	超时、重试、结构化证据	q18_ext/ verified_loop.py
Ex07	Issue 质量门禁	复现要素、退出状态	q19_ext/ issue_linter.py
Ex08	Review 质量门禁	级别、风险、动作、验证	q20_ext/ review_linter.py
Ex09	训练可复现性审计	固定种子、逐张量比较	q21_ext/ reproducibility_audit.py
Ex10	梯度诊断与裁剪	范数、有限值、clip	q22_ext/ gradient_diagnostics.py
Ex11	检查点往返验证	state_dict, CPU 加载	q23_ext/ checkpoint_roundtrip.py

统一验证

运行powershell -File scripts/verify_all.ps1 会检查Ruff、Wheel、隔离安装、q10 测试、q11 协作规则、q12 训练及全部11 个拓展实例。完整记录保存在docs/verification-log.txt，结尾为ALL WEEK 3 CHECKS PASSED。

3.1 Ex01: Wheel 元数据检查器

3.1.1 实例分析

Wheel 本质上是带规范目录的ZIP 文件。脚本在不安装包的情况下读取METADATA、WHEEL 和entry_points.txt，可在发布前发现发行名、版本、兼容标签或入口脚本错误。

```
with zipfile.ZipFile(path) as archive:
    metadata = email.message_from_bytes(archive.read(metadata_name))
    entry_points = dict(parser.items("console_scripts"))
    return {"name": metadata["Name"], "tag": wheel_metadata["Tag"]}
```

执行结果

对q09 Wheel 的检查结果为7 个归档成员，名称和版本正确，标签为py3-none-any，入口准确映射到greetlab.cli:main；1 项测试通过。

3.2 Ex02: 一次性环境Wheel 冒烟测试

3.2.1 实例分析

脚本使用TemporaryDirectory 与venv.EnvBuilder 创建一次性环境，使用-no-index -no-deps 安装指定Wheel，再从临时目录运行控制台命令。测试结束后环境自动回收，不污染日常解释器。

```
venv.EnvBuilder(with_pip=True).create(root)
subprocess.run([python, "-m", "pip", "install",
                "--no-deps", "--no-index", wheel], check=True)
completed = subprocess.run([command, "--name", student_id], cwd=root)
```

执行结果

隔离环境输出Hello, 24020007086!，工具报告isolated install: PASS。这证明安装产物和入口可用，而非本地源码偶然可导入。

3.3 Ex03: 公共API 兼容性门禁

3.3.1 实例分析

脚本用AST 提取模块顶层且不以下划线开头的函数与类，对比发布前后的集合。新增符号仅报告，删除符号则返回1，从而在构建新版本前暴露潜在破坏性变化。

```
return {node.name for node in tree.body
        if isinstance(node, (ast.FunctionDef, ast.ClassDef))
        and not node.name.startswith("_")}
removed = sorted(before - after)
```

执行结果

测试样例保留stable、删除removed、新增added，工具准确给出两个差集，1 项测试通过。

3.4 Ex04: 智能体补丁范围门禁

3.4.1 实例分析

工具先把Windows 和POSIX 分隔符统一，再将修改文件与允许路径前缀比较。这能把“只修改q10”转化为可执行约束，防止智能体顺手改动根目录配置或其他作业。

```
prefixes = [normalize(p).rstrip("/") + "/" for p in allowed]
if not any((path + "/").startswith(prefix) for prefix in prefixes):
    bad.append(path)
```

执行结果

输入两个q10 文件和根目录README 时，检查器只报告README.md 越界；仅输入允许文件时输出patch scope: PASS, 1 项测试通过。

3.5 Ex05: 编程智能体任务契约

3.5.1 实例分析

JSON 契约强制包含目标、约束、测试命令和允许文件。校验器还拒绝绝对路径与`..`，避免任务范围逃逸。相比自然语言中的“尽量少改”，这些字段可被程序检查。

```
REQUIRED = {"goal", "constraints", "test_command", "allowed_files"}
if path.is_absolute() or ".." in path.parts:
    errors.append(f"unsafe allowed path: {raw}")
```

执行结果

q10 示例契约输出**contract: PASS**；测试确认缺少测试命令且允许父目录时会同时报告两类错误，1 项测试通过。

3.6 Ex06: 有界验证循环

3.6.1 实例分析

工具为每次命令保存序号、退出码、耗时和合并输出，成功立即停止；超时转换为124，尝试次数由参数限制。这样失败不会无限重试，人工也能看到每次验证发生了什么。

```
for number in range(1, attempts + 1):
    completed = runner(command, capture_output=True, timeout=timeout)
    history.append(Attempt(number, completed.returncode, seconds, output))
    if completed.returncode == 0:
        break
```

执行结果

模拟验证依次返回1 和0 时只执行两次；真实命令一次成功并输出JSON 记录**returncode: 0**，1 项测试通过。

3.7 Ex07: Issue 可复现性检查

3.7.1 实例分析

检查器要求文本出现环境、复现、期望、实际、命令和退出状态；未知环境还必须明确写“待确认”。它不替代人工判断，但能过滤只有情绪、没有证据的缺陷描述。

```
REQUIRED_MARKERS = ("environment", "reproduce", "expected", "actual")
if "exit status" not in text:
    errors.append("missing observable exit status")
```

执行结果

q11 材料输出 **issue quality: PASS**；完整样例通过，而“Windows 上运行不了”会触发至少4项缺失提示，2项测试通过。

3.8 Ex08: 代码评审意见质量检查

3.8.1 实例分析

规则要求评审包含 **Blocking**、**Suggestion** 或 **Nit** 之一，并明确具体风险、建议动作和验证条件。这样作者能判断优先级，也知道满足什么条件才能合并。

```
if not any(label in text for label in SEVERITIES):
    errors.append("missing severity")
if not has_risk or not has_action or not has_verification:
    errors.append("review is not actionable")
```

执行结果

q11 评审输出 **review quality: PASS**；可执行评审通过，含糊的“这里写得不好”会缺少级别、动作和验证条件，2项测试通过。

3.9 Ex09: PyTorch 可复现性审计

3.9.1 实例分析

脚本以相同种子从头训练两次，不只比较打印值，而是使用`torch.equal` 逐张量比较两个`state_dict`，并要求最终损失完全一致。

```
first, first_loss = train_once(seed)
second, second_loss = train_once(seed)
exact = all(torch.equal(first[name], second[name]) for name in first)
```

执行结果

实际输出`exact_state_match: true`、`loss_match: true`，两次最终损失均为 1.5897×10^{-7} ，1 项测试通过。

3.10 Ex10: 梯度诊断与裁剪

3.10.1 实例分析

在`backward()` 后调用`clip_grad_norm_`，其返回值保留裁剪前范数，便于观察训练是否曾出现过大梯度；同时检查所有范数和损失均为有限值。

```
loss.backward()
norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
observed.append(float(norm))
optimizer.step()
```

执行结果

初始及最大梯度范数为14.2632，最终梯度范数约 2.45×10^{-6} ，最终损失约 1.23×10^{-12} ；学习到`weight` 约5、`bias` 约2，1 项测试通过。

3.11 Ex11: PyTorch 检查点往返验证

3.11.1 实例分析

检查点同时保存模型参数、优化器状态和步数。加载时指定`map_location="cpu"`与`weights_only=True`，再将参数装入全新模型，比较加载前后完整预测张量。

```
torch.save({"model": model.state_dict(),
           "optimizer": optimizer.state_dict(), "step": 100}, path)
payload = torch.load(path, map_location="cpu", weights_only=True)
restored.load_state_dict(payload["model"])
```

执行结果

生成的临时检查点为2133 bytes，恢复步数为100；预测逐元素完全一致，最大绝对差为0，1项测试通过。临时目录退出后自动清理。

4. 总结与个人体会

4.1 课堂实验（q09–q12）

- q09 让我认识到“源码能运行”和“软件能交付”是两回事。只有构建Wheel、在干净环境安装、并离开源码目录执行入口，才能真正证明打包边界正确。
- q10 把智能体使用变成红灯、最小修复、diff 审查和绿灯复测的闭环。智能体给出的改动必须接受测试与范围约束，不能只看文字解释是否合理。
- q11 表明工程沟通本身也是技术产物。环境、复现命令、期望、实际、风险和动作越具体，接手者越能直接执行，而不是重新猜测问题。
- q12 使训练循环的状态变化更清晰：梯度必须每轮清空，评估必须关闭梯度记录，随机种子和数值阈值共同构成可复现证据。

4.2 课后练习（Ex01–Ex11）

- 打包类练习把Wheel 从“生成的文件”扩展为可检查、可隔离安装、可执行验证的发布产物；API 门禁进一步把兼容性风险前移到发布之前。
- 智能体类练习把允许文件、任务契约、超时、重试和退出码组合成边界明确的自动化流程。我体会到可靠智能体的核心不只是生成代码，而是持续产生可审阅证据。
- 协作类练习将课堂写作规则变成Issue 和Review 检查器。自然语言不能完全自动判定，但最小信息要素可以形成门禁，从而减少低质量交接。
- PyTorch 类练习分别验证随机性、梯度和持久化：相同种子要逐张量一致，训练过程要监控梯度有限且收敛，保存后的模型要在CPU 上恢复出完全相同的预测。
- 全部11 个实例都由同一入口重复执行，并保留真实日志。这使结果从“曾经跑通”升级为“可再次运行、可定位失败、可由他人复核”。